

PyCore + ExCore: System Architecture

Component map for the research paper

PyCore systems notes
docs/paper/systems/architecture_overview.tex

August 2026

Abstract

PyCore is a research CPU whose native ISA is a filtered CPython 3.14 bytecode subset, implemented as a multi-cycle SystemVerilog hart with tagged 132-bit architectural values. ExCore is a companion RV32 multicycle hart that services *recoverable* container and helper traps in firmware under an exclusive shared-dmem ownership protocol. Together they form an asymmetric two-core machine: a bytecode fast path in hardware, and a firmware completion path that finishes the trapped semantic effect rather than merely retrying. This note decomposes the system into the components we propose as primary sections of the upcoming research paper, and provides an architecture diagram that places those components in relation to one another and to the roadmap (BIOS boot, on-device tokenizer, self-hosted `compile()`).

Contents

1	Thesis in one paragraph	2
2	Paper component map	2
2.1	Why these cuts (not a file tree)	2
3	System architecture diagram	3
4	Core control FSM	3
5	Component digests	5
5.1	Tagged bytecode ISA	5
5.2	PyCore microarchitecture	5
5.3	Tag-routed execute fabric	5
5.4	Object & container model	5
5.5	Asymmetric two-core design	5
5.6	Ownership & completion protocol	5
5.7	Container co-design	6
5.8	CALL FSM & frames	6
5.9	Exceptions & iteration	6
5.10	Image fidelity & tooling	6
5.11	Firmware builtins & ROM	6
5.12	Roadmap: toward self-hosting	6
6	Suggested paper narrative flow	6
7	Reference map	7

1 Thesis in one paragraph

Executing CPython bytecode as a first-class ISA forces microarchitecture to absorb language semantics that conventional CPUs push into compilers and runtimes: typed values, open-addressed dicts, call binding, exception tables, and iterator protocols. PyCore keeps the high-frequency typed paths in a multi-cycle bytecode hart; ExCore absorbs capacity-changing and $O(n)$ container work (and selected helpers) behind a mailbox and memory grant, under a **complete-the-semantic-effect** contract. The research claim is that this split—tagged bytecode ISA + asymmetric exception core + CPython image fidelity—is a practical way to study and accelerate Python-shaped workloads without reimplementing all of CPython’s object protocol in silicon.

2 Paper component map

Table 1 is the proposed section skeleton. Each row is a *paper-worthy* component: novel enough to stand alone, and load-bearing enough that later sections depend on it. Sibling systems notes already exist for some rows (e.g. the CALL FSM); others should grow into their own notes as they stabilize.

#	Paper section	Core idea
1	Tagged bytecode ISA	Native CPython 3.14 units as instructions; 132-bit {tag[3:0], value[127:0]} entries co-located in the RF.
2	PyCore microarchitecture	Multi-cycle, non-pipelined control (S_FETCH→DECODE→EXEC→MEM→WB) plus specialized FSMs (S_CONTAINER, S_CALL, trap marshal); see Figure 2.
3	Tag-routed execute fabric	Tag decode, BOOL/INT/FLOAT promotion, INT ALU / mul / div / FPU / complex, trap policy for unsafe tags.
4	Object & container model	Bump heap; stable LIST/DICT/SET objects with relocatable buffers; OBJECT kinds; contamination bit for bulk ops.
5	Asymmetric two-core design	Bytecode hart + RV32 ExCore; recoverable vs fatal trap taxonomy; private IMEMs, shared dmem.
6	Ownership & completion protocol	Mailbox handshake; exclusive memory grant; COMPLETED / RETRY / FATAL; “finish the effect, don’t just resize.”
7	Container co-design	Hash/probe/eq and spare-capacity append on PyCore; grow / extend / mid-delete / bulk merge on ExCore.
8	CALL FSM & frames	Shared binder for CALL / CALL_KW / CALL_FUNCTION_EX; dmem frame stack; method / builtin / type arms.
9	Exceptions & iteration	Exception-table field on code objects; exc-info stack; GET_ITER / FOR_ITER; protocol CALL pause/resume.
10	Image fidelity & tooling	compile() graph → tagged imem/dmem; boot record; fidelity boundary (bytecode-identical, layout not C-struct).
11	Firmware builtins & ROM	ROM-seeded pure-Python builtins; LEGB-B; hardware BI_* fast paths vs CODE_OBJECT frames.
12	Roadmap: self-hosting	Code ROM+RAM, BIOS, exec/eval, tokenizer (Plan 1); parser/AST/codegen/compile() (Plan 2).

Table 1: Proposed research-paper sections derived from the as-built system and documented direction.

2.1 Why these cuts (not a file tree)

A paper organized as “RTL modules we happen to have” would bury the interesting claims. The cuts above follow *research seams*:

- **ISA vs microarchitecture** — tags and opcodes are the contract; the FSM is how we implement them.

- **Language semantics in hardware vs firmware** — the ExCore boundary is the central systems contribution.
- **Fidelity vs completeness** — image tooling locks bytecode fidelity today; Plans 1–2 describe the path to host independence.
- **Hot path vs protocol path** — CALL/containers stay native; `__iter__`/`__next__` and exception handlers reuse the same CALL machinery via pause/resume.

3 System architecture diagram

Figure 1 is the single overview figure intended for the paper’s architecture section. Solid arrows are data/control paths in the as-built two-core top (`pycore_excore_system`). Dashed boxes are roadmap surfaces that already have planning contracts (`planning/code_loading_bios_tokenizer_plan.md`, `planning/native_compiler_plan.md`).

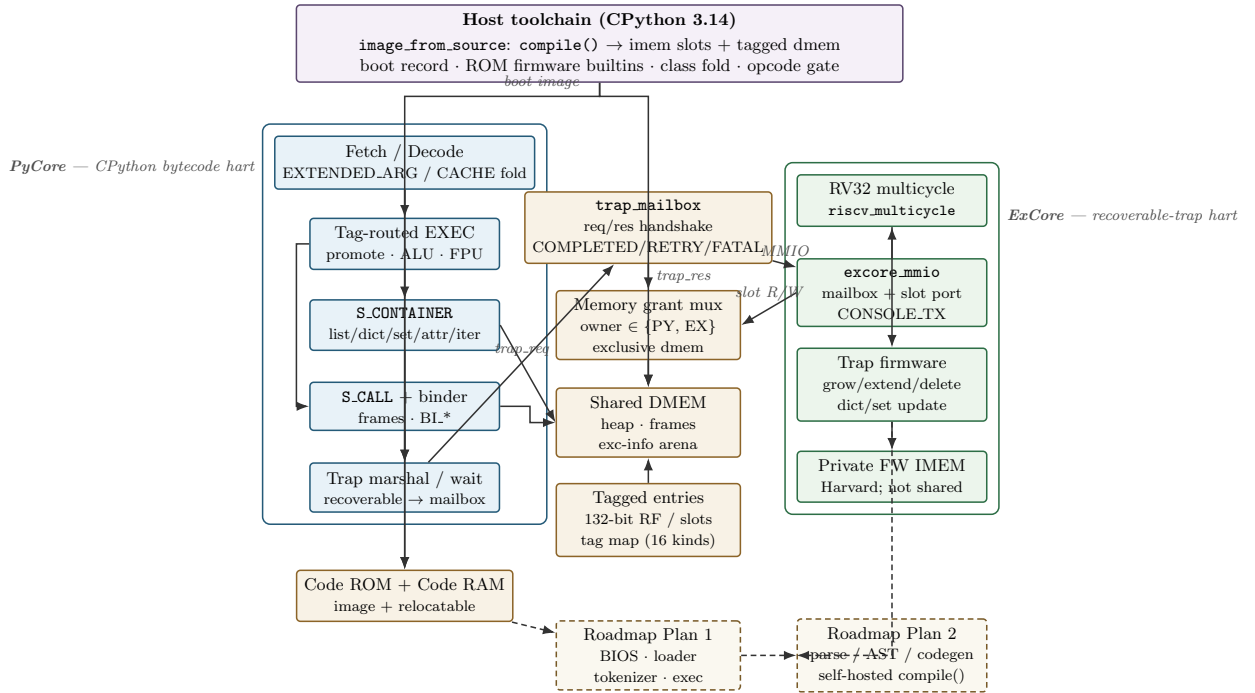


Figure 1: PyCore + ExCore system architecture. PyCore executes tagged CPython bytecode; recoverable traps cross the mailbox into ExCore firmware while PyCore is frozen and ExCore holds the exclusive dmem grant. Dashed nodes are planned surfaces toward on-device compilation.

4 Core control FSM

Figure 2 is the top-level `state_r` automaton in `pycore_core.sv` (4-bit encodings in parentheses). Exactly one instruction is in flight: there is no forwarding or hazard logic. Language-dense opcodes leave the classic `FETCH`→`DECODE`→`EXEC`→`MEM`→`WB` loop for specialized states.

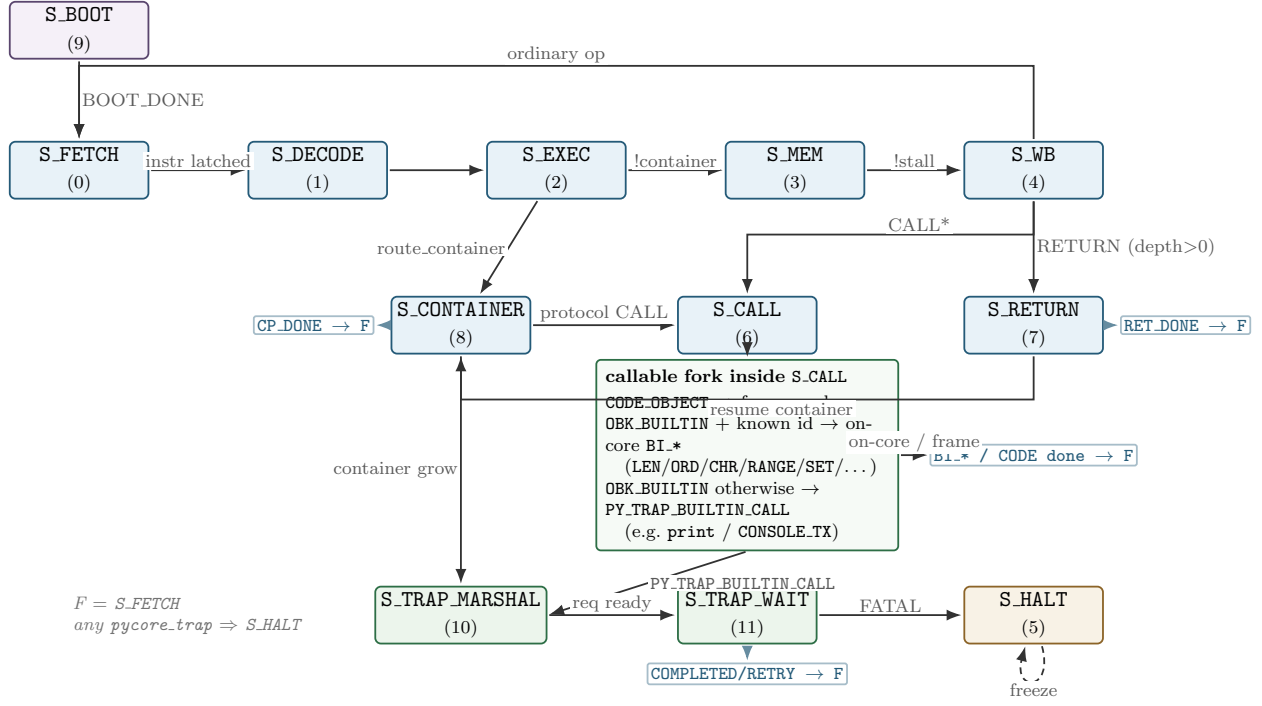


Figure 2: PyCore top-level control FSM (`state_r`). Solid arrows are local control transfers; dashed tags return to `S_FETCH`. Inside `S_CALL`, `CODE_OBJECT` takes the frame path and `OBK_BUILTIN` either completes on-core (`BI_*`) or raises `PY_TRAP_BUILTIN_CALL` into the ExCore marshal/wait path (e.g. `print`).

Staging roles.

State	Enc	Role
<code>S_BOOT</code>	9	Walk boot record; latch globals/builtins; enter fetch.
<code>S_FETCH</code>	0	<code>pycore_fetch</code> : fold <code>EXTENDED_ARG</code> , skip <code>CACHE</code> .
<code>S_DECODE</code>	1	Drive RF read addresses; latch operands.
<code>S_EXEC</code>	2	Tag-routed ALU/FPU/branch; hold on multi-cycle units.
<code>S_MEM</code>	3	PTR load/store via <code>pycore_mem_stage</code> .
<code>S_WB</code>	4	RF write + TOS update; redirect on taken branch.
<code>S_CONTAINER</code>	8	Heap ops, names/consts, attr, iter, raise; own TOS.
<code>S_CALL</code>	6	Binder + frame push; phase 13 builtin fork (<code>BI_*</code> or trap).
<code>S_RETURN</code>	7	Frame pop; reload caller <code>co_consts/co_names</code> .
<code>S_TRAP_MARSHAL</code>	10	Assert <code>trap_req</code> ; freeze RF/dmem.
<code>S_TRAP_WAIT</code>	11	Apply <code>trap_res</code> ; COMPLETED/RETRY→fetch, FATAL→halt.
<code>S_HALT</code>	5	Terminal freeze on any fatal trap.

Routing rules that matter for the paper.

- **Container bypass.** `route_container` (decode `is_container`, or list `NB_INPLACE_ADD`) leaves `S_EXEC` for `S_CONTAINER` and never visits `S_MEM`/`S_WB`.
- **Call after writeback.** Plain `CALL*` / `RETURN_VALUE` still take `S_WB`, then enter `S_CALL` or `S_RETURN`.
- **Builtin fork (phase 13).** In `S_CALL`, an `OBK_BUILTIN` handle either runs an on-core `BI_*` fast path (`LEN`, `ORD`, `CHR`, `RANGE`, `SET`, ...) and exits `CALL_DONE`→`S_FETCH`, or raises `PY_TRAP_BUILTIN_CALL` (e.g. `print`) into `S_TRAP_MARSHAL`→ExCore. ROM `CODE_OBJECT` builtins take the normal frame path instead.

- **Recoverable handoff.** A container or builtin arm sets `trap_marshal_pending_r` before its done phase; exit goes to `S_TRAP_MARSHAL` instead of `S_FETCH`.
- **Protocol pause/resume.** `GET_ITER/FOR_ITER` on `OBJECT` set `container_call_pending_r` → `S_CALL`; the outer return restores container context instead of fetching.

5 Component digests

The digests below are intentionally short: enough for a paper outline, not a replacement for `pycore/docs/architecture.md` or sibling systems notes.

5.1 Tagged bytecode ISA

Every architectural value is `{tag[3:0], value[127:0]}`. Tags cover `CONTROL`, `INT` (i64 fast path), `FLOAT`, `COMPLEX`, `BOOL`, `ITER`, `TUPLE`, `SHORT/LONG_STR`, `MUT_COLLEC`, `OBJECT`, `RANGE`, `CODE_OBJECT`, `TOMBSTONE`, and reserved slots. Instruction memory stores CPython code units one-for-one (including `CACHE` and `EXTENDED_ARG`); fetch folds them at runtime. This is the ISA claim: Python bytecode is the machine language.

5.2 PyCore microarchitecture

One instruction in flight; no forwarding or hazard logic. Figure 2 shows the top-level `state_r` staging. Specialized states absorb language density: `S_CONTAINER` for heap ops, `S_CALL/S_RETURN` for frames, `S_TRAP_MARSHAL/S_TRAP_WAIT` for ExCore handoff, `S_BOOT` for the boot record. Harvard: private imem (ROM+RAM), shared dmem bank tiled for retargeting.

5.3 Tag-routed execute fabric

`pycore_tag_decode` selects promotion and unit; unsafe tags trap rather than inventing a full object protocol in the ALU. Largest intentional semantic deviation: `INT` overflow wraps at 64 bits.

5.4 Object & container model

Bump-pointer heap; `LIST/DICT/SET` use stable object headers and relocatable tables/buffers (CPython-shaped split). `OBJECT` kinds carry `ob_head` (`INSTANCE`, `TYPE`, `BOUND_METHOD`, `BUILTIN`, `EXCEPTION`, ...). Bulk ops honor a contamination bit: uncontaminated paths may trap to ExCore; contaminated (`OBJECT`-key) paths stay on PyCore.

5.5 Asymmetric two-core design

ExCore is not a second Python hart: it is an RV32 firmware engine for recoverable traps (list grow/extend/delete, dict/set grow and update, builtins such as print via console, slice helpers, ...). Fatal traps still halt via `pycore_trap`. Private instruction memories; only the data heap is shared.

5.6 Ownership & completion protocol

On `trap_req` complete, grant flips to ExCore; on `trap_res` complete, grant returns to PyCore. PyCore is frozen (no dmem/RF activity) while ExCore owns memory. Handlers answer **COMPLETED** after finishing the semantic effect (e.g. grow *and* append), avoiding a second ownership round-trip. `RETRY` exists for future “emulate from scratch” handlers; live handlers complete.

5.7 Container co-design

On PyCore	On ExCore
Hash + rich equality	List grow / extend / mid-delete
Open-addressed probe / contains	Dict grow + rehash + store
Spare-capacity append; last-el delete	Set grow / update
Empty LIST_EXTEND no-op	Dict update / merge (uncontaminated)

5.8 CALL FSM & frames

Documented in `docs/paper/systems/call_fsm.tex`. Shared binder unifies positional, keyword, and starred shapes against `co_varnames` / `defaults` / `*args` / `**kwargs`. Frames are simple dmem push/pop descriptors (ring-buffer spill remains attic).

5.9 Exceptions & iteration

Code objects carry a raw `co_exceptiontable` field; host tooling parses CPython varints to slot ranges. `GET_ITER`/`FOR_ITER` cover native containers and `OBJECT` protocol via pause, `S_CALL`, then resume; `StopIteration` exhausts without a user `try`.

5.10 Image fidelity & tooling

“Identical to `compile()`” means same bytecode units, consts, names, and nested code objects in tagged encoding—not byte-exact CPython C structs. Deprecated `preprocess.py` is fixture-only; production is `image-boot`.

5.11 Firmware builtins & ROM

Boot builtins dict mixes `OBK_BUILTIN` hardware ids and ROM `CODE_OBJECTS` under `pycore_firmware/builtins`. `LEGB` ends at builtins; public `print` is ROM wrapping `_bi_print` / console MMIO.

5.12 Roadmap: toward self-hosting

Plan 1: relocatable code RAM, Python BIOS, `exec/eval` on precompiled code, on-device tokenizer (PyPy DFA port). Plan 2: LL(1) parser, AST, symbol table, codegen, assembler, bootstrap fixpoint. Paper discussion should separate *host independence* from *language completeness*.

6 Suggested paper narrative flow

1. Motivate bytecode-as-ISA and the cost of full object-protocol silicon.
2. Present the tagged ISA and image-fidelity contract (sections 1, 10).
3. Walk the PyCore microarchitecture and execute fabric (sections 2, 3).
4. Introduce the object/container model as the workload that forces asymmetry (section 4).
5. Reveal ExCore, ownership, and the completion contract (sections 5–7) as the systems contribution.
6. Deep-dive CALL and exceptions/iteration as language-faithfulness case studies (sections 8, 9).
7. Close with builtins/ROM and the self-hosting roadmap (sections 11, 12), then evaluation (CPO, trap rates, two-core differentials).

7 Reference map

Artifact	Role
pycore/docs/architecture.md	Living two-core architecture
pycore/docs/tags.md	Tag payloads
pycore/docs/object_model.md	OBJECT / builtins decisions
pycore/docs/dict_excore.md, set_excore.md	Container split
excore/docs/mmio_map.md	Mailbox / slot / console
docs/paper/systems/call_fsm.tex	CALL subsystem note
planning/code_loading_bios_tokenizer_plan.md	Plan 1
planning/native_compiler_plan.md	Plan 2